ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/344722762

Lean Comic Gan (LC-GAN): A Light-Weight GAN-Based Architecture Leveraging Factorized Convolution by Depthwise Separable, Pointwise CNN and Teacher Model Forced Distilled Fetch Forw...

Preprint · October 2020

DOI: 10.13140/RG.2.2.36258.86729

CITATIONS
0

READS
311

1 author:

Kaustav Mukherjee
Optum Health

3 PUBLICATIONS **0** CITATIONS

SEE PROFILE

All content following this page was uploaded by Kaustav Mukherjee on 18 October 2020.

The user has requested enhancement of the downloaded file.

Lean Comic Gan (LC-GAN): A Light-Weight GAN-Based Architecture Leveraging Factorized Convolution by Depthwise Separable, Pointwise CNN and Teacher Model Forced Distilled Fetch Forward Perceptual Style Loss Aimed to Capture Two Dimensional Animated Style Photos using Smartphone Camera and Edge Devices

Kaustav Mukherjee

***Abstract*—In this paper, we propose a solution to design a light-weight GAN-based filter for smartphone cameras, Raspberry PI 4 equipped with PiCam and similar edge devices. This GAN-based camera filter will carry out neural style transfer to input images and impart a 2D animated comic movie-style look and texture to it. This will help the 2D animation artist to design or create new characters from real life person's images or even create scenes from real life images. Our network reduces the density of the architecture by using Depthwise Separable Convolution which does the convolution operation on each of the channels separately, then utilises a point convolution using 1×1 kernel. This reduces the number of parameters substantially and makes our model light-weight. This paper aims to reduce the cost in terms of time and human effort required to create 2D animated movies. The generator's model fits in a size of 12.2 MB which makes it a suitable option to be used as chipset embedded model and act as filter for smartphone cameras or edge devices like Raspberry Pi 4, NVIDIA Jetson NANO etc. Existing design and architectures are more resource intensive compared to our proposed architecture, with the model size close to 170 mb. Owing to the use of high resolution input and bigger convolution kernel size, our model produces richer resolution pictures with 6 times fewer number of parameters while training on a dataset of 518 images. The salient feature about LC-GAN's training approach is the usage of Teacher-Student Paradigm for faster training of the model.

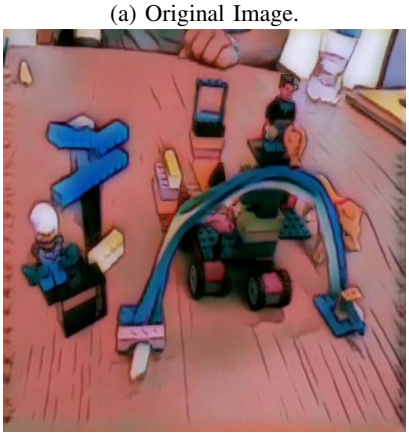
Keywords—2d Comic Stylisation from Camera Image using GAN,Creating 2D Animated Movie Style Custom Stickers From Images,Depth-wise Separable Convolutional Neural Network for light-weight GAN architecture for EDGE Devices,Gan Architecture for 2D Animated Cartoonizing Neural style

I. Introduction

THE methodology described in this paper helps to create a real-time,distributed,scalable deep-learning or machine-learning inference framework on Kubernetes architecture.The real-time straming of data is carried out by Apache Kafka whereas Apache Nifi enables to build the end to end pipeline with minimum amout of coding saving significant time and cost.The use of Kubeless aids serverless inferencing using fucntion as a service.

A. Cartoon Rendering using LC-GAN example:

(a) Original Image.



(b) Comic Style Image.

Fig. 1: Comic Stylization Example 2 Using Light-Weight LC-GAN.

Kaustav Mukherjee is working with Huawei Technologies,India,Bangalore. (phone:+919886462190,email[office:kaustav,mukherjee@huawei.com], email[personal]:oracler4284@gmail.com)

B. Applications in Industry:

1) *Camera or video filters for mobile phone users of Instagram like applications:*
   a) LC-GAN generator acts as custom filters for mobile cameras to capture Comic Style pictures or videos. Due to the light-weight size of LC-GAN's model, it is eligible for native integration with a smartphone camera interface and provide the filter control to the user as an additional camera mode.
2) *Animated movies and 2D animation arts industry:*
   a) In the animated film industry, a substantial amount of time is taken in generating sketches for animation movies. A lot of manual labour is spent in designing background scenes, characters etc. This technology will enable to generate animation arts which will relieve a substantial amount of pressure from the artists which will result in cost-cutting and time-saving.
   b) LC-GAN's architecture has been designed for use as a native filter for smartphone camera and enables animated movie makers to convert mainstream movies to their animated version in minutes.
3) *Gaming industry for generation of new characters for games based on movies or sports:*
   a) Many games are developed based on some prior released movie or sports like FIFA, The Chronicles Of Riddick etc. Their themes are essentially based on movies or sports. The looks of the characters designed for these games need to have a greater resemblance with that of the actual movie star or the sportsman. It will assist the gaming industry to easily generate characters for games based on movies or sports, by applying our filter to the movie frames.

C. Related State of the Art:

1) *CycleGAN:*
   a) CycleGAN, a GAN-based Neural Style Transfer variant which can learn to translate images without paired training data to overcome the limitations of one-by-one pairing of pix2pix [17] in image translation. CycleGAN can automatically translate two given unordered image sets X and Y. The architecture of CycleGAN uses two adversarial losses for X →Y and Y →X to check the quality of generator's output. CycleGan [18] was originally introduced keeping in mind to translate high quality pictures. Moreover in CycleGAN [18], the primary philosophy is proper replacement of the characters, e.g. horse getting replaced by a zebra. The animal needs to replaced exactly and essentially in real-world high-quality images of real scenes. CycleGan needs higher training time and also higher number of parameters. CycleGAN [18] needs to train two GAN [14] models for bidirectional mappings. The content loss can be a single based on l2-loss between content and generated images.
   b) CycleGAN's [18] model size is more than 100 MB. This makes it resource intensive for edge-based systems like RaspBerryPI 4, as there are multiple models deployed at the same time in a maximum memory limit of 4 GB.
2) *CartoonGan:*
   a) CartoonGAN [10] is a GAN based [14] approach for neural transfer of 2D animated movie styling to input images. It is not a pairing algorithm. It takes two sets one set of cartoon images and one set of real images. It uses a GAN [14] based architecture with one generator and one adversary network. It maps images from the photo manifold into the cartoon manifold while keeping the content unchanged.
   b) It consists of an initialisation phase to improve the convergence of the network to the target manifold.
   c) It uses two loss functions generative loss and adversarial loss. In the generative network, to cope with substantial style variation between photos and cartoons, we introduce a semantic loss defined as an l1 sparse regularisation in the high-level feature maps of the VGG network [10]. In the discriminator network, it uses an edge-promoting adversarial loss for preserving clear edges [10].
   d) Though CartoonGAN [10] is capable of style transfer from 2D animated photos to real-world photos it seemed resource intensive for edge devices with size almost over 100 MB for the model parameters. This amount of memory just for a single AI application seemed quite heavy for resource constrained devices like smartphones, Raspberry PI4 etc.

II. Methods

A. Lean Comic Gan (LC-GAN):

1) *Data-sets for generating 2D animated filter for camera shots:*
   a) For training our models we used clips from the movie Heman Battlecat. We trained the model with only 518 images using a very shallow neural network for style transfer.

B. Machine setup used for training:

   a) GPU TITAN XP (11 GB)
   b) CPU 2*Xeon 2670 (16 Cores/32vCores)
   c) RAM 128 GB

C. Salient features of this LC-GAN design which improves the current state of art design:

1) *Factorzied convolution [7], [2] for model pruning:*
   a) Standard convolution:
   b) A standard convolutional layer takes as input a $DF \times DF \times M$ feature map and produces a $DF \times DF \times N$ feature map G where DF is the spatial width and height of a square input feature map1, M is the number of input channels (input depth), DG is the spatial width and height of a square output feature map and N is the number of output channel (output depth) [20]. The diagram of the standard convolution is as the figure below.
   c) The output feature map for standard convolution assuming stride one and padding is computed as [20]:

$$G_{k,l,n} = \sum_{i,j,m} K_{i,j,m} \cdot F_{k+i-1,l+j-1,m}$$

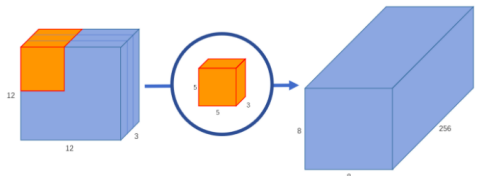


Fig. 2: Standard Convolution.

Fig. 3: Depth-wise Separable Convolution for Parameter Reduction.

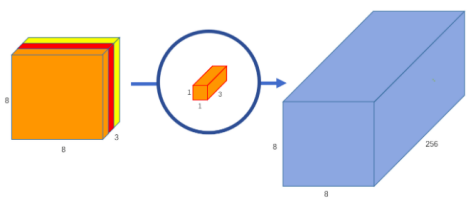


Fig. 4: Point Convolution to induce non-linearity.

d) Standard convolutions have the computational cost of [20]:

$$DK \cdot DK \cdot M \cdot N \cdot DF \cdot DF$$

e) where the computational cost depends multiplicatively on the number of input channels M, the number of output channels N, the kernel size Dk × Dk and the feature map size DF × DF [20].

f) Model pruning by leveraging depthwise convolution:

g) The diagram of pruning the model using depthwise and pointwise convolution is given below [20]:

h) Concept of Depthwise Convolution followed by Pointwise Convolution [20]:

i) When a d-dimensional kernel can be expressed as the outer product of d vectors, one vector per dimension, the kernel is called separable [2]. It is equivalent to compose d one-dimensional convolutions with each of these vectors. The composed approach is significantly faster than performing one d-dimensional convolution with their outer product [2].

j) Depthwise separable convolutions, which is a form of factorized convolutions, separates a standard convolution into a depthwise convolution and a 1×1 convolution called a pointwise convolution [7].

k) Depthwise convolution with one filter per input channel (input depth) can be written as [20]:

$$G'_{k,l,m} = \sum_{i,j} K'_{i,j,m} \cdot F_{k+i-1,l+j-1,m}$$

l) where K' is the depthwise kernel of size $D_k \times D_k \times M$, where the m_{th} filter in K' applied to the m_{th} channel of F will produce the m_{th} channel of the output feature map of G' [20].

m) Depthwise separable convolutions cost is the sum of the depthwise and 1 × 1 pointwise convolutions [20]:

$$DK \cdot DK \cdot M \cdot DF \cdot DF + M \cdot N \cdot DF \cdot DF$$

n) By expressing convolution as a two step process of filtering and combining, we get a reduction in computation of:

$$\frac{DK \cdot DK \cdot M \cdot DF \cdot DF + M \cdot N \cdot DF \cdot DF}{DK \cdot DK \cdot M \cdot N \cdot DF \cdot DF} = \frac{1}{N} + \frac{1}{{D^2}_k}$$

o) The main purpose of a 1x1 kernel is to apply non-linearity [7], [2]. Non-linear layers expand the possibilities for the model, as is what generally makes a "deep" network better than a "wide" network. In order to increase the number of non-linear layers without significantly increasing the number of parameters and computations, we can apply a 1x1 kernel and add an activation layer after it. This helps give the network an added layer of depth. This is one trade-off where we increase the depth instead of the channel to lower the number of parameters but still be able to capture useful features due to addition depth [7], [2].

2) *Art of teacher guided distilled fetch forward perceptual style loss for faster style extraction [5]:*

a) The Philosophy:

b) If a child learns everything from raw data then what is the role of teacher ?

c) Human beings do not learn everything from raw data. Suppose a child is studying a subject, he can opt to read it from a book directly. This kind of learning we can interpret as learning from raw data. But in real life, we have seen a child does not learn directly from a book from its first day alone. Rather, he is given a teacher or a guide and this is the sole purpose why a child is sent to school. That means before trying to learn directly from a book, the child learns from the teacher and then reads the book for reference or it may the reverse, like learning a bit from the book and then taking the teacher's assistance.

d) Who is the guide or the teacher and who is the student or the child?

e) We have seen in many instances that after a certain age and maturity, a person can learn directly from books without the need to go to a guide or teacher as frequently.

f) With age, we have experience which is the data collected and our brain labelled over time with the consequences learnt from our actions.

g) Our critical thinking increases with age.

h) In terms of deep learning, an aged or matured model is just like an aged human being which will have more activations.This implies 3 things:

i) a. More layers

j) b. More neurons/hidden layer units

k) c. More data

l) A sufficiently large model well-trained for an optimum number of iterations with huge amount of diverse data will play the role of a teacher model or a guide model.

m) The student/child/lean model is the smaller model which will have very few layers and neuron/hidden units. It is deliberately made light-weight so that it can execute smoothly on resource-constrained devices. This light-weight, shallow model will play the role of a child model.

n) For a Neural Style Transfer model, just like when a child first studies from some books, we train the child model with some initial number of iterations for the initialization.

o) Once the Child model initialization is done, for each iteration, the Child model generates an image and the heavy-weight pre-trained teacher model generates an image. Both the teacher and child generates a stylised output of the same input image. Then, the output images are fed to an encoder for extraction of the semantic features. The encoder's outputs are collected and a weighted l1 loss is computed between the teacher generated and the child generated images.

p) Teacher guided distilled fetch forward perceptual style loss for faster style extraction and faster learning:

q) In this paper for calculating the DPFSL, the Lean Comic GAN generator output and Teacher Generator output is compared on the same ground truth image allowing the model to converge in 900 iterations with 518 images.

r) The faster convergence is shown in the figure below:

(a) Generator Output after 907 iterations without DPFSL.

(b) Generator Output after 907 iterations without DPFSL.

Fig. 5: Strong effect of teacher guided distilled fetch forward perceptual style loss for faster convergence and better style transfer.

3) Teacher guided distilled fetch forward perceptual style loss (DFPSL):

a) Calculating distilled fetch forward perceptual style loss using teacher guidance:

b) We create a super-deep model with optimum number of hidden units per layer. Then we train this model on a vast amount of data until the loss converges to a satisfactory point. After that, the trained model starts to generate satisfactory 2D animated images; we mark this model as the Teacher model.

c) Then we create a shallow model with very few layers. We train this mini-model/student model for about 500 iterations with approximately 500 style images and content images, respectively. After 500 iterations, the student generator model starts generating images which looks like content images.

d) At this point we checkpoint the Student model and induce the Distilled Fetch Forward Perceptual Style Loss. After this step for every iteration of training, we make the Student model generate a 2D-styled image from a source content image and at the same time we let the frozen Teacher model generate its version of a well-defined 2D-styled image from the same source content image. Both the images from the teacher and the student generator is fed through a VGG_{16} encoder to extract the semantic features. Then the distance between the semantic feature-set is calculated.

4) Vanilla Batch Normalization Process:

a) Batch normalization is one of the most exciting recent innovations in optimizing deep neural networks and it is actually not an optimization algorithm at all. Instead, it is a method of adaptive re-parameterization, motivated by the difficulty of training very deep models [2].

b) If H be a minibatch of activations of the layer to normalize, arranged as a design matrix, with the activations for each example appearing in a row of the matrix and where μ is a vector containing the mean of each unit and σ is a vector containing the standard deviation of each unit. The Batch Normalization formulae is represented by the formulae below [2].

$$H' = \frac{H - \mu}{\sigma}$$

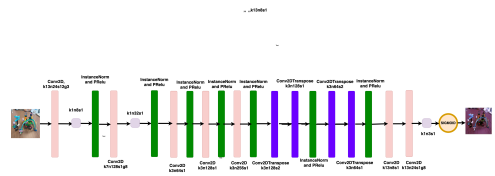


Fig. 6: Teacher Generator Model.

5) PitFalls of Batch Normalization:

a) Batch normalization comes as trade-off between the ease of learning and making the lower layers useless or linear. Batch Normalization normalizes the first and second order statistics which in turn deprives the lower layer from learning any non-linear useful representations [2]. By the property of linear functions and Universal Approximation Theorem, as the final layer of the network is able to learn a linear transformation we can may remove all linear relationships between units within a layer, who provided the inspiration for batch normalization. Eliminating all linear interactions is much more expensive than standardizing the mean and standard deviation of each individual unit.

6) Our Approach of Learnable Parameterized Batch Norm:

a) To keep the expressive power of the network, it is common to replace the batch of hidden unit activations H with parameterized or weighted normalized values [2]. Rather than using the normalized output of batch norm i.e. H' it is parameterized like $\gamma H' + \lambda$ rather than simply the normalized H'. The variables γ and λ are learned parameters that allow the new variable to have any mean and standard deviation. This new parametrization has different learning dynamics. In the old parameterization, the mean of H was determined by a complicated interaction between the parameters in the layers below H. In the new parametrization, the mean of $\gamma H' + \lambda$ is determined solely by λ. The new parameterization is much easier to learn with gradient descent.

7) Use of parameteric ReLU as learned rectifier:

a) Parametric ReLU is a learned parametric activation unit that helps in better feature extraction [21].

b) Here, y_i is the input of the nonlinear activation f on the i_{th} channel, and a_i is a coefficient controlling the slope of the negative part. The subscript i in a_i indicates that we allow the nonlinear activation to vary on different channels. The activation function is defined as [21]:

$$f(y_i) = \begin{cases} y_i & \text{if } y_i > 0 \\ \alpha_i y_i, & \text{if } y_i \leq 0 \end{cases} \tag{1}$$

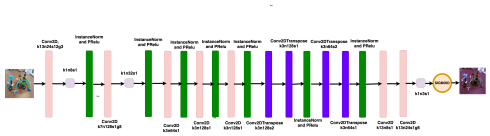


Fig. 7: Student Generator Model.

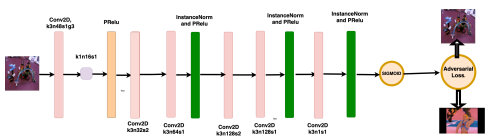


Fig. 8: Adversarial Network Model.

D. LC-GAN Architecture and Training Strategy:

a) As stated in section LC-GAN uses two models.

1) Teacher Model Architecture:

a) The Teacher model architecture diagram is shown in the Fig. 6 below.

b) The Teacher model is trained using approximately 2000 data points for 50 iterations in order to generate 2D animated styled images from content images provided as inputs. In practice, the Teacher model can be of any size provided the training server supports it.

c) Once the Teacher model is trained and it generates satisfactory styled images we construct a shallow Student model.

2) Student Model Architecture:

a) The Student model architecture diagram is shown is figure below.

b) The Student model is trained for 500 iterations with 518 images for content and style respectively. After that it starts to generate some images which looks like the content images but the style transfer is very minimal. At this point we checkpoint the model.

c) Then at each iteration we use the trained Light-Weight Generator to generate image from an input and we make the Teacher Generator to generate image from the same input fed to the Light-Weight Generator. The loss is calculated as the distance between the encoded outputs of the Teacher model and the Light-Weight model.

3) Adversarial Model Architecture:

a) The Adversarial network is shown in the figure below.

E. Loss Function:

a) There are two Loss components in this architecture; Generator Loss and Adversarial Loss.

1) Generator Loss:

a) The Generator Loss component τ is given by the following formulae:

$$\tau(G,D) = \underset{x\in C, y\in I}{E}[D(x) + \underset{y\in X}{E}(1 - D(G(y)]$$

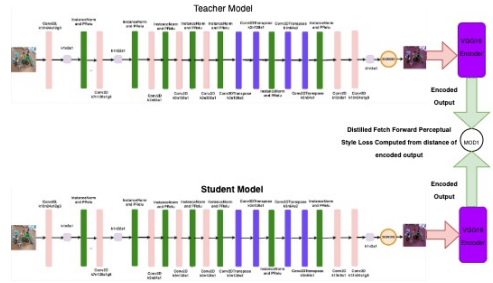


Fig. 9: Distilled Fetch Forward Perceptual Style Loss Computation Architecture.

image Size	Runtime	Teacher Model Size with Loss	Student Model Size with Loss	Cartoon GAN Model Size with Loss
256*256	15 fps	12 mb	15 mb	170 mb
512*512	7 fps	12 mb	15 mb	170 mb
64*64	18 fps	12 mb	15 mb	170 mb

Fig. 10: Performance and Parameter Size Comparative Analysis of LC-GAN with competitors.

b) where, C is the set of 2D animated images and I is the set of real images taken from the camera.

c) The Content Loss Ψ is given by the following formulae:

$$\Psi(G,D) = \underset{z\in I}{E}(\| \underset{16}{VGG}(G(z) - \underset{16}{VGG}(z)) \|)$$

d) where VGG_{16} is used to capture the features in a low-dimensional space. X is the set of images taken from the camera and G(z) is the stylised images output that is obtained from the generator output.

e) Distilled Fetch Forward Perceptual Style Loss Π is given by:

$$\Pi(\underset{teachermodel}{G}, G) = \underset{z\in I}{E}(\| \underset{16}{VGG}(\underset{teachermodel}{G}(z) - \underset{16}{VGG}(G(z))) \|)$$

f) where I is the set of images taken from camera. VGG_{16} encoder to capture the features in a low-dimensional space.

g) The total loss of the generator Υ is given as:

$$\Upsilon = \tau + \frac{1}{10^3} \times \Psi + 2 \times \frac{1}{10^3} \times \Pi$$

2) Distilled Fetch Forward Perceptual Style Loss Computation Architecture:

a) The distilled fetch forward perceptual style loss computation diagram is given in Fig. 9 below:

3) Adversarial Loss

a) The Adversarial Loss Λ is composed of two components which is given as:

$$\Lambda(G,D) = \underset{x\in C}{E}[D(x) + \underset{y\in X}{E}(1 - D(G(y)]$$

b) where D is the discriminator function, G is the generator function, C is the dataset consisting of 2D animated images and X is the dataset consisting of images taken from smartphone camera.

F. Experiments:

1) Sylized Images using LC-GAN:

(a) Original Image. (b) Comic Stylised Image.

Fig. 11: Comic stylization experimental results set 1 using light-weight LC-GAN.

(a) Original Image. (b) Comic Stylized Image.

Fig. 12: Comic stylization experimental results set 2 using light-weight LC-GAN.

2) *Comparison of the loss minimization to the number of iterations of the same model with and without Teacher Model Forced Distilled Fetch Forward Perceptual Style Loss:*

```
USING GPU
torch.float32
torch.Size([2, 1, 128, 128])
Epoch 1501/1501 | d_loss 0.4067 | g_loss 4.5308 | total no. of losses 1
USING GPU
torch.float32
torch.Size([2, 1, 128, 128])
Epoch 1501/1501 | d_loss 0.3984 | g_loss 5.1371 | total no. of losses 1
USING GPU
torch.float32
torch.Size([2, 1, 128, 128])
Epoch 1501/1501 | d_loss 1.8867 | g_loss 3.8967 | total no. of losses 1
USING GPU
torch.float32
torch.Size([2, 1, 128, 128])
Epoch 1501/1501 | d_loss 0.2793 | g_loss 5.1951 | total no. of losses 1
```

(a) Model without Distilled Fetch Forward Perceptual Style Loss.

```
USING GPU
torch.float32
torch.Size([1, 1, 128, 128])
Epoch 593/593 | d_loss 1.5234 | g_loss 2.9662 | total no. of losses 741
USING GPU
torch.float32
torch.Size([1, 1, 128, 128])
Epoch 593/593 | d_loss 0.7315 | g_loss 3.5059 | total no. of losses 741
USING GPU
torch.float32
torch.Size([1, 1, 128, 128])
Epoch 593/593 | d_loss 0.7490 | g_loss 3.9061 | total no. of losses 742
USING GPU
torch.float32
torch.Size([1, 1, 128, 128])
Epoch 593/593 | d_loss 0.9943 | g_loss 3.2602 | total no. of losses 742
```

(b) Model with Distilled Fetch Forward Perceptual Style Loss.

Fig. 13: Loss Comparsison.

3) *Richer resolution and smoother texture of stylized images with lighter model and less training time leveraging LC-GAN's approach:*

   a) The cartoon GAN uses a model which is close to 170 MB in size compared to this paper's approach of LC-GAN which is 12.2 MB in size. According to the paper of CartoonGAN The training data contains real-world photos and cartoon images, and the test data only includes real-world photos. All the training images are resized and cropped to 256 $\times$256 [10].

   b) As a 256×256 image is fed as input to the Neural Network of CartoonGAN [10] outputs get blurry when magnified at 512×512.

   c) This clearly infers that this paper's architecture of LC-GAN produces double resolution richer output with a model size which is 15 times smaller and trained on ten times less data compared to the state-of-art CartoonGAN [10] which sizes close to 170 MB and trained on close to 5000 images.

   d) The output image quality difference of LC-GAN and state-of-art CartoonGAN [10] taken at 512×512. The below pictures shows the higher detailing of LC-GAN compared to CartoonGAN's generator output.

(a) CartoonGAN output getting blurred at high resolution.

(b) Richer Resolution of LC-GAN generator output showing better details

Fig. 14: Richer Resolution of LC-GAN compared to Cartoon-GAN.

III. Conclusion

a) Our future roadmap is to integrate distributed machine learning traning framework to facilitate scalable training on kubernetes.We have also got a plan of integrating data cataloging by means of graph triplet representation to facilitate metadata as a service.This will enable to capture and relate context metadata aong with technical metadata for superior data unification and governance.

References

[1] Saining Xie, Zhuowen Tu. *Holistically-Nested Edge Detection*. https//arxiv.org/abs/1504.06375.

[2] Ian Goodfellow, Yoshua Bengio and Aaron Courville. *DeepLearning* [*The MIT PRESS.*]

[3] Milan Sonka The University of Iowa, Iowa City, Vaclav Hlavac Czech Technical University, Prague, Roger Boyle Prifysgol Aberystwyth, Aberystwyth. *Image Processing, Analysis, and Machine Vision*.

[4] C. Bucilua˘, R. Caruana, and A. Niculescu-Mizil. *Model compression. In Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '06, pages 535–541*. New York, NY, USA, 2006. ACM.

[5] Geoffrey Hinton, Oriol Vinyals and Jeff Dean. *Distilling the Knowledge in a Neural Network*. https//arxiv.org/abs/1503.02531v1.

[6] Antonio Polino, Razvan Pascanu and Dan Alistarh *MODEL COMPRESSION VIA DISTILLATION AND QUANTIZATION*. arXiv1802.05668v1 [cs.NE] 15 Feb 2018.

[7] François Chollet *Xception Deep Learning with Depthwise Separable Convolutions*. https//arxiv.org/abs/1610.02357.

[8] M. Wang, B. Liu, and H. Foroosh. *Factorized convolutional neural networks*. arXiv preprint arXiv1608.04337, 2016.

[9] M. D. Zeiler and R. Fergus. *Visualizing and understanding convolutional networks.*. In Computer Vision–ECCV 2014, pages 818–833. Springer, 2014

[10] Yang Chen, Yu-Kun Lai and Yong-Jin Liu. *CartoonGAN Generative Adversarial Networks for Photo Cartoonization.*. CVPR 2018.

[11] L. Gatys, A. Ecker, and M. Bethge. *Image style transfer using convolutional neural networks.*. In IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pages 2414–2423, 2016.

[12] L. A. Gatys, A. S. Ecker, and M. Bethge. *Texture synthesis and the controlled generation of natural stimuli using convo- lutional neural networks.*. arXiv preprint arXiv1505.07376, 12, 2015.

[13] L. A. Gatys, A. S. Ecker, M. Bethge, A. Hertzmann, and E. Shechtman. *Controlling perceptual factors in neural style transfer.*. In IEEE Conference on Computer Vision and Pat- tern Recognition (CVPR), 2017.

[14] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. *Generative adversarial nets.*. In Advances in Neural Information Processing Systems 27, pages 2672–2680. 2014.

[15] C. Ledig, L. Theis, F. Husza r, J. Caballero et. al. *Photo-realistic single image super-resolution using a generative adversarial network.*. In IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.

[16] J. Johnson, A. Alahi, and L. Fei-Fei. *Perceptual losses for real-time style transfer and super-resolution.*. In European Conference on Computer Vision, pages 694–711, 2016.

[17] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, Alexei A. Efros. *Image-to-Image Translation with Conditional Adversarial Networks*. arXiv1611.07004v3 [cs.CV] 26 Nov 2018.

[18] Jun-Yan Zhu, Taesung Park, Phillip Isola, Alexei A. Efros. *Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks*. https//arxiv.org/pdf/1703.10593.pdf.

[19] Chi-Feng Wang *A Basic Introduction to Separable Convolutions*. https//towardsdatascience.com/a-basic-introduction-to-separable-convolutions-b99ec3102728.

[20] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, Hartwig Adam *MobileNets Efficient Convolutional Neural Networks for Mobile Vision Applications*. arXiv1704.04861v1 [cs.CV] 17 Apr 2017.

[21] Kaiming He, Xiangyu Zhan, g Shaoqing Ren, Jian Sun *Delving Deep into Rectifiers Surpassing Human-Level Performance on ImageNet Classification*. arXiv1502.01852v1 [cs.CV] 6 Feb 2015

View publication stats